Neural network representation of Boolean circuits

Shouvik Sur

2026-10-09

Table of contents

1 Neural representations of 2-bit logic gates **1**
1.1 SLP representation of Linear Threshold Functions 2
1.1.1 `AND` 2

2 Neural representations of 3-bit logic gates **3**

1 Neural representations of 2-bit logic gates

Most standard 2-bit gates can be represented by a single layer perceptron (SLP). These include the `AND`, `OR`, `NAND`, and `NOR` gates. This can be understood geometrically by considering the 2-bit dataset $\mathcal{S} = \{0,1\}^2$ as coordinates of the corners of a square. These four gates classify $\mathcal{S}$ by "drawing" *one* decision boundary.

By contrast, the classification performed by `XOR` will require two decision boundaries. Consequently, `XOR` cannot be represented by an SLP and the minimal neural network requires one hidden layer with two neurons. This results in a depth 2 neural circuit for representing and learning `XOR`.

Interestingly, since `NAND` is a universal gate, it is possible to represent `XOR` by a circuit of four `NAND` gates. This circuit of logical gates, however, has depth 3. Here, we show how to negotiate this mismatch of circuit depth, and in the process develop insights into compressing logical-gate circuits to match the minimal MLP implementations of the same function. We anticipate such compressions are maximal for tasks that are "hard" for logical gates but "easy" for MLPs due to the inherent non-linearity of the latter.

1.1 SLP representation of Linear Threshold Functions

A linear threshold function (LTF) performs a linear transformation on its inputs and outputs a binary result based on a threshold condition. `AND`, `OR`, `NAND`, and `NOR` gates are examples of LTFs. In order to verify the statement, let us obtain the functional representation of these gates. We will consider a function $f(\vec{x}) = \Theta(\vec{w} \cdot \vec{x} + b)$ with $\vec{w}$ and b being parameters, Θ the Heaviside step function, and the two dimensional vector $\vec{x} \in \mathcal{S}$. Note that $f(\vec{x})$ is an SLP with a single neuron and two inputs, characterized by the weights w_1 and w_2, and the bias b. By utilizing the truth table of the respective gates, we can place restrictions on $\vec{w}$ and b as follows.

1.1.1 `AND`

Truth table:

x_1	x_2	$x_1 \wedge x_2$
0	0	0
0	1	0
1	0	0
1	1	1

Thus, $f(\vec{x})$ must satisfy

- $f((0,0)) < 0 \Rightarrow b < 0$.
- $f((0,1)) < 0 \Rightarrow w_2 + b < 0 \Rightarrow w_2 < -b$.
- $f((1,0)) < 0 \Rightarrow w_1 < -b$.
- $f((1,1)) > 0 \Rightarrow w_1 + w_2 > -b$.

Since these conditions are mutually compatible, `AND` can be represented by the two-neuron SLP $f(\vec{x})$. A simple example of

In `PyTorch` this minimal multilayer perceptron (MLP) is defined as

```
class XORModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.main=nn.Sequential(
            nn.Linear(2, 2),
            nn.ReLU(),
            nn.Linear(2, 1),
            nn.Sigmoid()
        )
```

```
    def forward(self, x):
        return self.main(x)
```

2 Neural representations of 3-bit logic gates

Generalizing beyond the 2-bit operations, here, we consider the `Toffoli` and `Fredkin` gates.